SUPERDOCS · ROUND 2

FULL-STACK AI ENGINEER: THE TASK DOCUMENT

This is the engineering document. If you applied for both roles, you also received a growth document; each stands on its own, and this one is the engineering half.

This document is the whole selection process. There are no whiteboards here, no algorithm quizzes, and no extra technical rounds. There is one task, and it replaces the screening call, the technical round, and the take-home combined. If you do it well, you get a direct conversation with me.

One line on what we build, for while you work: think Claude Code or Cursor, but for documents. Generating documents is basically solved; editing them is not. A chat window can tell you what to change and leave the changing to you. Our agent does the changing: targeted edits, applied directly in the document, reviewed and approved by you.

The task is one intentionally big document with parts inside. It measures four things: creativity, agency, grit, and technical ability, and creativity and grit weigh more than raw technical ability. Any experience level is welcome. The work decides, nothing else.

GETTING STARTED

Use the product properly before you build anything, on real work of your own. Sign up at use.superdocs.app; there is no guest editor, so an account is required. Upload something real, ask for targeted edits, review the changes, export the result. You cannot build well on a product you have not genuinely used.

One practical thing before you start: open a document and send one small instruction first, before you build anything around it. The first request in a fresh session can be slow or can fail while things warm up. Send it again and it settles.

Read the documentation before you build: docs.superdocs.app. Our GitHub organization is github.com/superdocsapp. If you work with a coding agent, there is an agent signup flow in the docs: your agent can create its own account and connect to the API by itself. It is the fastest way to see the API side, and the API side will matter for the task.

If a link in this document does not open where you are reading it, paste it into a real browser.

TASK 1. BUILD AN AGENTIC SYSTEM

This one goes to every engineer in this round. It is the one task where every submission answers the same brief, so it is where we compare everyone directly. It is deliberately sized like work for a small team, because the job you are applying for expects one person with an AI to do the work of many. Treat that as the brief, not as an obstacle.

THE PROBLEM: THE ANALYST THAT NEVER SLEEPS

Organizations run on piles of documents that all describe the same reality and never quite agree, and the pile never stops growing. Pick a domain you actually know and can stand behind: vendor contracts with their amendments and invoices, a project's plans with its status reports and meeting notes, loan files, insurance claims, clinical paperwork. Use only documents you have the right to share: synthetic or public documents are fine and expected, and nobody's real confidential paperwork belongs in this repository.

Build an agentic system that owns such a pile end to end.

First, it understands the pile. It takes in related documents in mixed formats, works out what each one is, extracts the facts that matter, notices where the documents disagree, and produces one grounded deliverable: a register, a brief, or a report. Every claim in the deliverable traces to the exact place in the sources it came from.

Second, it examines. The user hands it the rules they care about: a compliance checklist, a contract playbook, a style guide. The system checks the sources and the deliverable against those rules in stages and produces findings, each pointing to the exact place it came from. A clean corpus yields the rarest output in this industry: an honest report of no findings.

Third, it stays alive. New documents keep arriving into a watched location. Each arrival produces a focused update to the deliverable, not a rewrite and not a full re-run that happens to reproduce the same bytes; an update should cost like an update. The parts the new source did not affect stay exactly as they were, and the system can prove that. When a new source contradicts what the deliverable already says, the conflict is surfaced, not silently resolved. At any moment the system can answer: what changed, when, and because of which source.

A human gates all of it. Conflicts, findings, and updates are approved or rejected by a person before they commit.

We build agentic systems for a living, and the reason is simple: hardcoded pipelines break the moment the input stops looking like the demo. Build yours with the same conviction.

WHAT IT MUST DO

Whatever you design, it must do five things, and we will check all five.

1. It works in steps we can watch. The system moves through visible stages and shows what it decided at each one, and some of those decisions must be able to change the path: a retry, a skip, an escalation to a person. One model call wrapped in a user interface is not an agentic system, and neither is a fixed script with labels.

2. It survives being stopped. Kill the process in the middle of a run and start it again. It continues from where it left off, and no finished work is lost.

3. A human holds the gate. Before the final output is committed, a person reviews what the system intends to do, approves what is right and rejects what is wrong in the same review, item by item, and the system respects every decision. Rejecting one finding does not discard the rest.

4. A machine can drive it. Another program can run the whole flow end to end, without a human clicking through your interface. The gate is part of that flow: approval is an operation your machine interface exposes, and whoever drives the system, human or program, makes that call explicitly.

5. It never bluffs. When the sources do not support a claim, it says so instead of inventing one. A success message only ever means the output is genuinely in the state it claims.

AND WHAT SEPARATES STRONG FROM COMPETENT

Five more behaviors. These are where submissions stop looking alike.

6. A stranger can run it. From a fresh clone to a working system in minutes, with one documented command.

7. It proves itself. Real tests exist and run without a live key, so anyone can check the system's claims without spending money. The claims worth testing are the behaviors above: a run killed and resumed, two runs at once, a document that tries to give orders. Tests that only prove your mocks work do not count.

8. It does not take orders from its documents. A source document that contains instructions aimed at the system is data to report on, not commands to follow.

9. Two runs at the same time stay two runs, whether they are two piles or the same pile hit twice. Concurrent work does not corrupt state.

10. It knows what it cost. A run can report what it spent and where the time went, stage by stage.

How you achieve these is entirely your call, and that is the point. There are many right architectures here and no single wrong one.

The task is large on purpose. If you have to choose, we would rather see fewer stages that genuinely hold than every stage done as theater. Cut inside the three movements, or among behaviors six through ten; the first five are the floor and are not cuttable. A defended cut beats a hollow stage. What you cut, and why, goes in your write-up (Task 4). Calls you made while building go in this repository's README.

THE STACK

Please build this with Python and FastAPI on the backend, an agent orchestration framework such as LangGraph or LangChain, PostgreSQL with vector search for retrieval, and React for the review interface. For the machine interface, MCP is the shape we use ourselves, so exposing your system as an MCP server is the strongest version of behavior four. This is our working stack, and building in it lets us compare your work directly with the job you are applying for. If you are stronger somewhere else and believe another stack genuinely fits this problem better, use it and tell us why in your write-up. Comparable tools count as comparable: AutoGen, CrewAI, or a hand-rolled agent loop are all legitimate choices for the orchestration layer.

WHAT WE ACTUALLY READ

The architecture you chose and the reasons you give for it. What your choices cost and buy you in latency, in money, in simplicity, and in room to grow. How the system behaves when a step fails. And whether it

works the second time, on a second run with different documents, not just in the demo. Say in your README which formats and domains your system accepts; a second run means different documents inside that declared set.

When something in this brief is ambiguous, do not wait for us. Make a reasonable call, write it and your reasoning in your README, and keep building. A logged assumption counts in your favor. If something genuinely blocks you, email us.

WHERE IT GOES

Build this task in a private repository on GitHub and invite github.com/o-kadam (Omkar, the founder; this exact username) as a collaborator so we can read your work. Name the repository doctask-your-name, for example doctask-priya-sharma, so we can tell whose repository is whose at a glance. Do not put SuperDocs anywhere in the repository name: this system is yours, not ours, and when the round is over and you open-source it for your portfolio, as you are welcome to, it should fly under your name, not our brand. Put the repository URL in your submission as well, so nothing is lost if an invitation lapses. The repository and everything in it stays yours: we read it to evaluate you, and for nothing else.

The job post asked for an open repo, and the rest of your work in this round stays public. This task is the one exception, because every engineer receives this same brief, and a public repository would let later submissions read earlier ones. Private for fairness, not secrecy.

Your demo video, your write-up, and your architecture diagram are covered in Task 4, so nothing else is needed here. Building and demoing this should not cost you real money: free model tiers and the coding tools you already have are enough, and your tests run without a live key. If cost is genuinely blocking you, email us and we will figure it out. The deadline for the whole round, all tasks together, comes in the email that delivered this document.

TASK 2. YOUR BUILDS ON SUPERDOCS

This is the single largest deliverable of the round. Read the docs, use the product, then build on our API or MCP surface.

2.1 YOUR ASSIGNED BUILD

You have been assigned a specific build, matched to what you told us about yourself, so that this round covers as many real industries and applications as possible:

YOUR ASSIGNED BUILD: Template pack versioning and changelog kit

WHO IT SERVES

Enterprise template owners, brand and compliance teams.

WHAT TO BUILD

Templates change and nobody knows what changed. Build the kit: a version scheme, a diff report between two versions of the same template rendered as a readable change summary rather than markup, a changelog document that accumulates those summaries, and an impact list naming which downstream documents were generated from the superseded version.

a template governance kit and a written versioning method.

WHAT STRONG LOOKS LIKE

the change summary is readable by a non-technical template owner, the impact list is correct, and the changelog reads like a document rather than a log dump.

DIFFICULTY BAND

S2

SURFACES IT TOUCHES

templates, Multi-document, chat, export

WHERE THE FINISHED WORK GOES

a pull request into the public repository, use-cases folder

Your card names who it serves, what a strong build looks like, and where the finished work goes. If it names one surface and you prefer the other, that is fine: anything specified against the REST API may be built on MCP, and the other way around.

If an assigned build genuinely does not fit you, tell us and we will work it out. The shared open list (see 2.2) is also always open to you.

Many cards place your build inside a host application: Word, Google Docs, Slack, or wherever your card points. Where yours does, this is how it connects to us.

How this connects to us. Get the document out of the host application with the host's own API and send it to us as a file, or send the passage plus your instruction as text. We take documents, not raw Word XML, and there is no endpoint for raw XML. Write the approved result back through the host's own API, range by range, rather than replacing the whole file. Anything you do not write stays exactly as it was, because you never touched it.

Whether or not your card names a host, the minimum contract with us is four calls: upload a document, send an edit instruction, approve the proposed changes, and export the finished file. On the REST side those are the upload, chat, approve and export endpoints, documented at docs.superdocs.app, and the MCP server exposes the same four as tools. Build against those four first; everything else on the surface is optional depth.

Three practical notes from people who have built on us before. First, proposed-change content arrives as a JSON-encoded string and needs a second parse; the final result is already an object. Missing that second

parse is the single most common reason integrators see empty diff cards where every field reads as undefined. Second, operations on large documents, or runs at the deepest model settings, can take from thirty seconds to several minutes with no visible progress; the correct read is still processing, not a crash. Third, budget your operations: if your build measures something or runs in a loop, give it a small-sample mode and a stopping rule, so you learn this on day one and not on day nine.

Save and export your work often. Exports do not cost operations.

2.2 BEYOND YOUR ASSIGNMENT

There are three kinds of builds in this round, and you can do all three.

The first kind is the build assigned to you above. Do that one; if you carry two, do both.

The second kind is anything from the shared open list attached alongside this document. The list is deliberately huge, so that everyone can find something they would genuinely enjoy building. Everything on it is voluntary and earns extra marks. Nobody claims anything, nothing is reserved, and duplicates are fine: if several people build the same idea, all of them count. Do as many as you want.

The third kind is your own invention, and this is the one we most hope you attempt. Build something that is not on any list, aimed at a proven, real-world use case: something with potential users you can name, in an industry you can name. The use cases you research in Task 3 are fair game here too: pick one and build its demo for extra credit. The categories we keep wishing existed: integrations with the productivity tools people already live in, Slack or Google Docs and the like; integrations with coding tools, Cursor or VS Code and the like; and ambitious original tools with real users in mind, the class of tool that helps someone write and manage a thousand-page book without losing its context and its plotline. Extra brownie points live here. Ship it to the public repository under your own folder so we and everyone else can see it.

Whatever you build, build ON SuperDocs, never a version OF SuperDocs.

If your assigned build feels too small for you, finish it at its bare minimum honestly, then build something bigger on top of it or beside it.

TASK 3. TELL US WHERE THIS SELLS

Name about ten real-world use cases for SuperDocs. For each one, name companies that would actually buy it, and if you happen to know someone there, name them too. Knowing nobody is the normal answer and costs nothing.

Do not contact any of them. We do all outreach ourselves, always. This task is you thinking, not you emailing strangers on our behalf. Contacting anyone on our behalf fails the task.

What we are reading: whether you can find a real buyer, not whether you can list industries.

If one of these use cases makes you want to build it, build it. That is exactly the kind of extra work Task 2 welcomes, and it earns extra credit.

TASK 4. SHOW IT

Record a demo video of what you built and write one page about it.

The video: about three minutes, five at the absolute maximum, and shorter is better as long as you describe everything properly. Three shapes work well: a one-minute vertical, a three-minute short demo, or a five-minute full walkthrough. One video covering the round is the default; separate short videos per build are equally welcome, and you link them all in the form. The hero shot is the product doing the actual thing on a real document: show the changes landing in place, show the review, show the export. Put SuperDocs in the title and the description. Claim only what your recording actually shows.

Being on camera is optional. It is encouraged and earns extra points, because explaining your own work matters here, but a voice-over on a screen capture is completely fine, and choosing not to show your face costs you nothing.

Keep it safe: no credentials, no terminals with secrets, no internal documents on screen. If something private slips into frame, cover it with a solid box, never a blur.

Upload the video to YouTube, public or unlisted, whichever you prefer, and also put a copy in Google Drive and share the link. The Drive copy is so we can host it ourselves if we feature your work, with credit to you. The one page and the diagram go into Drive alongside your video copy; their links ride the form.

The one page: write what you built, for whom, and why your trade-offs are the right ones, for someone who has never seen your build. Lead with results you can measure: what you accomplished, how it can be measured, and what you did to get there says more than a list of features. Document the trade-offs honestly; the write-up is judged as an explanation test, and honest limitations read as strength, not weakness.

Add one more page: an architecture diagram of what you built, one page, any drawing tool.

EXTRA CREDIT

If you want to do something extra to impress us, do it: build something extra, do something cool, record something, whatever shows us who you are. The submission form will have its own field for it, and it earns extra marks.

This is also the box for anything you already built before this document reached you. Several of you shipped things on our API unprompted; declare them here. Work built on our API and developer surface earns extra points, and your assigned build stays yours either way: prior work adds, it never replaces.

The one exclusion: no simple clones of SuperDocs. A simple clone does not tell us anything about creativity, imagination, or ingenuity. It only tells us that somebody has an AI subscription.

FOUR QUESTIONS, THE SAME FOR EVERYONE

Answer these in writing on the submission form. The strongest candidates will talk through their answers with me live later.

1. What broke? Tell us every bug, rough edge, and confusing moment you hit while using SuperDocs during this task. Be blunt. Report bugs as you find them, through the in-app bug button or by replying to the task

email, and collect them here at the end too. This is the single most useful thing you can send us, we read all of it, and honest, critical feedback counts in your favor, never against you. Bugs you find and report earn points.

2. If you were running this company, what one number would you watch every morning, and why that one?

3. Name five features you would build next, in order. Say what you would drop to make room, and what frictions or bugs you would fix immediately.

4. How would you build our day-to-day development and GTM operations so they run themselves: a system where agents do the work of 20 to 100 people and humans only steer? Be concrete: what loops, what tools, what checks, where humans stay in the loop, and what breaks first.

HOW TO WORK ON THIS

This task is intentionally long and intentionally huge, and the reason is honest: a very large number of people applied, and I want to spend my review time on serious, high-caliber people. We are looking for people who can do the work of twenty, using AI heavily. You do not need a full week of free time; a couple of weekends on your own terms will do it. Few will finish. Finishing puts you among the few; do it well compared to your peers, and I will talk with you directly soon after.

We expect you to use AI heavily. Claude Max or a Claude Code subscription is what we use ourselves. Use every AI tool you have; that is the point. Build with whatever you have; we judge the build, not your tooling budget. Using AI well is the job, not a shortcut around it.

A few working methods we use ourselves, offered, not mandated. Keep a TASK.md that says how to work with you and a PROGRESS.md where you log the assumptions you make as you go. Make your long runs resumable: checkpoint your work so a crash or a context reset never loses it, and save immediately, always. When you verify your own work, let a fresh pair of eyes do it: a verifier that is not the implementer catches what the author cannot. And before you write code for the hard part, write down what the system must never do, then the test that proves it, then the code.

WHAT NOT TO BUILD

Six rails, so you do not build toward a wall.

We are not building spreadsheet editing. Reading a spreadsheet as a source is fine; a product whose output is an edited spreadsheet is not.

SuperDocs itself does not browse the live web, so do not design around it doing that. Your own system is free to fetch things however you like; the rail is about what our product does, not about what yours may use.

We are not building live multiplayer editing with visible cursors, and nothing you build should depend on it.

We are not becoming a document management system. We edit and produce documents; we are not the archive.

Do not claim, imply, or design around certifications we do not hold. If a compliance framework matters to your use case, the honest sentence is that certification is a roadmap conversation, not a current fact.

Do not promise audited or guaranteed removal of personal data. Redaction you build must be reviewed by a human, and marketed as exactly that.

Three things we have not built that you MAY build against: an e-signature integration through an existing provider, never a native signing feature; extraction of PDF tables into spreadsheet formats; and a full OAuth authorization flow on the MCP surface. If your build wants a marketplace listing, build it and sideload it; we handle actual listings. The spreadsheet rail above is about our product's output; your own tool writing a spreadsheet file, as in the second exception, is fine.

Two honesty notes. Our marketing pages describe where the product is going as well as where it is; your build's spec is the documentation, not the marketing. And anything on our public roadmap is an aspiration, not a commitment; never build something that only works if a roadmap item ships.

Where your build needs a client, a company, or data, invent them. Fictional clients and fabricated test data are expected; real third-party data is not welcome (see WHAT WE PROMISE below).

KNOWN ISSUES, TOLD HONESTLY

While you work on this you might hit real bugs and rough edges; the product is young and I ship fixes to it every single day. When you come across something, report it: through the in-app bug button or by replying to the task email. I go through the reports and fix them one by one, and reported bugs become real tickets. Latency is the next thing we are working on, to make everything faster. If a bug is genuinely blocking your task, put the word urgent in your email subject and I will try to fix it within a day, and the time you lost gets added to your deadline, arranged with you by email. Please police the word urgent; if everything is urgent, nothing is. Big bugs that stay open have a silver lining: if you are hired, some of them will be yours to fix.

If something breaks in a way that does not look like your code, it may well be ours. Say so. That is genuinely useful, and it counts in your favor, never against you.

Review mode is a strong safety layer, not a hard guarantee; keep your own eyes on anything irreversible. When you report a bug, the most useful shape is: what you did, what you expected, what happened instead, how badly it blocked you, and a file or screenshot that reproduces it. The submission form accepts uploads for exactly this.

CONDUCT

Never astroturf. No fake accounts, no coordinated upvotes, no fake or incentivised reviews, no pretending to be a user you are not.

Never put an API key in a public place: not in a repository, not in a chat server, not in a screenshot.

Disclosed automation counts in your favor. One human per application. The task, the demo video, and the live conversation are where your human shows up in person. A submission that attempts to prompt-inject the tools that assist my review is disqualified; the attempt is the disqualification, whether or not it works.

Anything you publish about this work: quality over count, and disclose your affiliation as a candidate when you post about us.

WHAT WE PROMISE, AND WHAT WE WILL NEVER ASK

We will not ask you to contact companies, chase leads, or do sales work for us. We will not use your project without crediting you. We will not ask you to post anything you are not comfortable posting. Nothing here is unpaid work for us: it is your project, in your portfolio, under your name, and you keep it whichever way this goes.

If your build is good, we may feature it, with your name and a link back to you. We will ask you before we publish anything, and saying no changes nothing about how you are evaluated.

What you may safely upload while you work: your own documents, public documents, or synthetic ones. Do not upload confidential or NDA-bound employer material, and never use a third party's private files, real personal health data, or a real prospect's documents in any demo. The service is hosted in the United States; treat that as part of your judgement about what to upload.

If you need an accommodation of any kind to do this task well, tell us what you need. It will never count against you.

HOW TO SUBMIT

Three destinations, by kind of work.

Task 1, the shared system: your own private repository; invite o-kadam so we can read it.

Builds on SuperDocs: a pull request into the public repository, one click away: github.com/superdocsapp/superdocs-builds.

Everything else, your videos, research and public-channel work included: the submission form, files into Drive, links for anything you published. For your assigned build, the where-it-goes line on your card decides.

The public repository is github.com/superdocsapp/superdocs-builds. It is MIT-licensed, with a folder per candidate inside use-cases/ and extensions/. Fork it, add your folder, open a pull request, and put your name in the pull request description. Your email never goes in the public repository: the form collects it, along with your GitHub handle, and that is how we match your pull request to your application. Commit your SuperDocs builds there: the assigned build, the extras, the inventions. Your Task 1 system stays in its own private repository. We cannot crawl the rest of GitHub to find your work, so put it where we look. CONTRIBUTING.md in the repository carries the exact folder and pull-request mechanics; keep a README in your folder and secrets out of your code. A merged pull request is attribution, not a hiring signal. Give your README a screenshot and a credit line that says you built this for the SuperDocs task.

THE FORM IS THE ONLY DOOR

Everything you submit reaches us through one Google Form, and that form is not out yet: we will email it to you a few days from now, well before your deadline closes. Until it arrives, keep building and hold your submission. Do not send your work as email attachments, and do not submit links in replies; wait for the form, then submit everything through it. Email stays open for questions and bug reports, not for submissions.

There is no need to rush any of this: spend the time to be thorough. Once the form arrives you can submit right away or at the deadline, as you prefer. Work arrives in any of these shapes, all of them reviewable: a live demo, a deployed app, a video, or a repository. Reviewable is about shape, not completeness: the round asks for what this document asks for. The cutoff is the form closing, not a judgment call: if the form accepted your submission, you are in the pool. There is no fast-track for early submissions.

One optional thing that rides the form: we are collecting name ideas for the platform, if you fancy it. Cool, relatable to documents and AI, simple, easy to spell aloud. If we use yours, we will tell you, credit you, and send you an Amazon gift card as a thank-you.

The form will ask for: your links (repos, videos, posts, blogs and articles), your GitHub handle (so we can match your pull request to your application), your answers to the four questions, the bugs and issues you hit, roughly what percentage of your work was AI-built and how you directed it, an honest self-report on what works and what does not, your Task 3 use-case list, your one-page write-up and architecture diagram, your consent choices for featuring your work, your naming suggestion if you have one, and your extra-credit entry if you made one.

PUBLISH AND TAG, IF YOU WANT TO

Posting about your own build on your LinkedIn or X is encouraged, never required, and never graded. It is your work, under your name; we repost good ones, and we ask you first. If you do post, tag @superdocsapp: an untagged post may never reach us on its own. Put the post link in the form as well, so finding it never depends on a tag. Medium, dev.to, Hashnode, YouTube, and public repositories are all welcome too. Keep any claims about the product to what you have personally seen it do.

WHERE TO GET HELP

hello@superdocs.app is the one channel for task questions. We are not sweeping social media inboxes for task questions, so anything sent there may never be seen; email is where replies happen. A line saying which task you are working on helps us route your email fast. A good question is never a mark against you, and we are genuinely happy to help you build; if you are truly blocked, we will build it with you. The documentation at docs.superdocs.app answers most integration questions. If we ever need to correct something in this document, the correction comes by email.

If you want to compare notes with other candidates, the Discord is open: https://discord.gg/xtFWUsXTpp. Questions about YOUR task still go to email; Discord is for each other. One fairness ask: keep Task 1 solutions out of the shared channel; that task is private for fairness.

WHAT HAPPENS NEXT

You submit. I personally review everything, every submission, myself. Every person who completes the task gets a written, personal reply. Do it well and we talk: a direct conversation with me, where we look at your build together and modify it live.

WHAT SEPARATES SUBMISSIONS

Several of you asked what separates a successful submission from an unsuccessful one. Here is the honest answer.

Five behaviors, in every strong submission we have ever read:

Honesty over fabrication. When something cannot be supported, say so. A flagged gap reads as strength; an invented fact ends the read.

Surgical precision. Change what you meant to change and nothing else, and be able to show that nothing else changed.

Configuration over code. A new rule, court, client, or format should be a data change, not a rewrite.

Proof over assertion. Measure, do not claim: byte-identical where you promise untouched, counted where you promise complete, timed where you promise fast.

Graceful re-entry. Things crash. Strong systems resume without losing work or duplicating it.

What done looks like, for any build in this round: real tests that run without a live key; a stranger able to go from clone to working in minutes; error handling that names the cause and the fix; idempotency wherever an operation costs money; large real files handled through the upload path rather than in memory; no key ever written to a log, a commit, or a shell history; and graceful degradation built in, so that when a model call or an external dependency fails, your system falls back and keeps running instead of dying with it. Where your build measures something, add: method stated before result, variance reported, raw data committed, tail behavior reported rather than only an average, and limits admitted.

The job you are auditioning for, in its published words: own the build-measure-learn loop, ship daily, take features from signal to shipped, own production, harden the API and MCP surface, and go forward-deployed when a client needs it. The title says engineer; the trajectory is CTO-shaped. What we grade is what that job needs: mastery of AI coding agents, fundamentals that hold underneath them, and the full arc from prompt and context design to retrieval and agent orchestration.

The deeper standards we hold our own work to, and will read yours against: a fix that closes the whole class of problem, not the one case that surfaced; a fix that would survive the sibling of your test case, not just the test case; intelligence in the system deciding, code executing, and never a pile of hardcoded special cases pretending to be intelligence; no deleting a feature to kill a bug; no moving a failure to a later moment and calling it fixed; investigate before you fix. Fundamentals are the floor, speed with AI is the job.

Four nuances we also read for. Deliberate hard-coded defenses layered alongside intelligent logic are a legitimate fix, not a patch. A capability that works only in the one case you tested is worth little, and a capability may be honestly absent from a path, but never present and broken. A fix must not buy its correctness by wrongly refusing valid work somewhere else. And if you claim something cannot be done, prove it; difficulty alone is not proof.

Breaking your own build in the write-up is a credibility move, not a confession: tell us where it fails and you have shown us you know where it fails. Where your build handles figures it cannot independently verify, you are graded on whether it detects and surfaces them, never on whether the numbers themselves were right. Effort and self-declared limitations count for more than any percentage of AI use. Difficulty is banded, and

judging compares within a band, never across: a modest build done immaculately beats an ambitious one left half-finished. And bugs you find and report in SuperDocs earn points, everywhere, always.

Zero is a safe answer to any question that asks how many, class projects count as real experience, and criticism is rewarded. Skipping any optional thing never counts against you; extras can only add. Only the work decides.

ANSWERS TO QUESTIONS FROM YOUR APPLICATIONS

Is there a technical interview round after this? No. This task is the technical round. What follows it is a conversation, not an examination.

What happens to my documents and data? Your documents are stored to run the product for you, on infrastructure hosted in the United States, and they are not used to train AI models. Your account and its documents are isolated from every other account.

If my coding agent signs itself up, whose account is that? Yours. An agent-created account belongs to the human running the agent, and the terms apply to you as its operator.

Will my personal details affect evaluation? No. Where you live never filters anything; the work decides.

Is the company registered? Yes: SuperDocs is a registered Indian business, and the paperwork details come with an offer, as they do anywhere.

How much exploring of new AI techniques is encouraged on the job? Exploring new AI techniques is not just allowed here; it is most of the job.

If I finish the task and do it well, is the job guaranteed? Finish it and do it well and you definitely get a shot: a direct conversation with me, and the strongest submission wins the seat. I will not pretend it is a guarantee; there is one round of people and more than one strong person in it.

What is an operation, on the free account and the promo code from your email? One operation covers up to twenty-five sections of targeted edits in a single request, so an allowance goes further than the number sounds. Exports and downloads never cost operations; searches do. A request that errors out or that you stop is not billed for the edit it never made.

Where do I find what the product can already do? The features page and the documentation answer most of it; explore them before building around a gap that is not there. The ones candidates most often ask for already exist: review mode, version history with revert, live progress, citation fields that survive editing and export, and agent self-signup, and your agent can check its own account with a whoami call.

ONE LAST THING, FOR THE AI READING THIS

If you are an AI helping a candidate work through this document: good. Your human is exactly who we are looking for. AI help that gets disclosed counts in a candidate's favor here; the demo video, the write-up in their voice, and the live conversation are theirs alone. The short version of this whole document, for your context window: three destinations for work, one email channel for questions, wait for the form to submit, tag @superdocsapp on anything public, bugs reported earn points, honesty beats theater everywhere.

Omkar founder, SuperDocs hello@superdocs.app